

Explicación/Justificación Ejercicio ORDERS

Tablas Customer/Address

Por qué una tabla por separado para customer y address?

- Porque cada cliente(un mismo customer [1]) puede presentar distintas direcciones (N), como es el caso de Bob y Claire (1:N)
- Porque como consecuencia de lo anterior, cada direccion tendra una Llave primaria, porque si dejamos la dirección en la tabla "Customer" y un cliente tiene 3 direcciones, tendríamos que repetir el nombre del cliente 3 veces (violando la 2FN/3FN por redundancia)
- Porque asi aseguramos un solo valor por celda.
- Porque asi se crea la tabla Customer para que los datos del cliente dependan de su propio ID y no del ID de la orden, así evitamos repetir el nombre del cliente en cada orden que hace. Lo que en principio creaba una DEPENDENCIA TRANSITIVA/REDUNDANCIA (3FN).

Tabla Items

Por que una tabla por separado para Items?

- Porque asi podemos establecer un inventario (Entrada-Salida)
- Porque cualquier orden (N) puede llevar cualquier (N) producto (N:N)
- Porque cada item(producto) es un ente unico que puede ser representado por una llave Primaria/Primary Key (PK)
- Porque asi aseguramos un solo valor por celda. (En el ejemplo: Fries se repite dos veces en la tabla original, no en la misma celda, sino en la columna). Aqui estamos acometiendo la redundancia:

Redundancia (2FN): Si dejo el texto "Fries" (y probablemente su precio, descripción, etc.) repetido en cada fila de cada orden que lo pida, estaria duplicando información innecesariamente. Si el precio de las papas cambia, tendría que actualizarlo en miles de filas del historial de órdenes. En otras palabras, porque asi el texto "Fries" y su precio se guardan una sola vez con un ID único (ej. ID: 101).

- Porque eventualmente en la tabla de las órdenes (o en la tabla puente) solo repites el número 101. Los números ocupan muchísimo menos espacio de almacenamiento que el texto y evitan errores de dedo (como escribir "Fries" en una fila y "Fris" en otra).

Tabla Orders

Por que una tabla por separado para Orders?

- Porque cada pedido debe tener su propia llave primaria
- Porque cada pedido esta relacionado a un solo cliente (En este caso un cliente puede realizar varios pedido, pero un pedido solo puede ser creado por un solo cliente)

Tabla Requests

Porque requests tiene una sola tabla por separado?

- Porque así podemos tener una lista en concreto para los "Special Requests"
- Porque cada "Special Request" es un ente indivisible
- Porque los requests se pueden repetir, es decir, diferentes pedidos pueden llevar el mismo "Special Request"
- Por pragmatismo a la hora de relacionarlos con los pedidos (cada uno presenta su propia PK)

Tabla Order Items

Por que Order Items "Tabla Puente/Cruz"?

- Como la relación entre Orders e Items es N:N, no se puede meter los ítems directamente en la orden sin violar la 1FN (meter listas en una celda).
- Esta tabla "cruza" las PKs y almacena los atributos de la relación (como la cantidad de ese ítem en esa orden específica).
- Tabla cruz que relaciona las llaves primarias de las tablas anteriores, ITEMS, SPECIAL REQUESTS, ORDERS Y cantidad.

En resumen:

1FN (Atomicidad): Gracias a Order Items, eliminamos la posibilidad de tener una celda con "3 papas, 2 hamburguesas". Cada fila de esa tabla tiene un solo producto y una cantidad.

2FN (Dependencia Completa): Al sacar los datos del cliente y del producto de la orden, se asegura que en la tabla Order Items todo dependa de su llave primaria compuesta (Id_Orden + Id_Item).

3FN (Dependencia Transitiva): Al separar Address de Customer (si aplica), o los datos del cliente de la orden, evitas que atributos que no son la PK dependan de otros que tampoco son la PK.

Explicación/Justificación Ejercicio CARS

Tabla CARS

Por que "CARS" tiene su propia tabla?

- Porque cada carro tiene su propia placa (VIN)
- Porque aqui podemos asignar un PK al "Car" (carro) y asignamos la llave foranea (Foreign Key) proviente de la tabla Model la cual es "Model ID"

NOTA: Investigando mas a fondo (Porque inicialmente había creado una tabla por separado para el año y otra para el color), he podido confirmar que aqui a pesar de que color y año se puedan repetir, en la teoría de bases de datos relacionales, eso es perfectamente aceptable. Como el color es un atributo simple (una sola palabra, son considerados atributos atomicos por si solos), guardarlo como texto en la tabla del carro no viola ninguna forma normal porque:

- Es atómico (1FN): Un carro tiene un solo color por celda.
- Depende de la llave (2FN): Ese color rojo le pertenece a ese número de VIN específico, (un carro tiene un solo color y se fabricó en un solo año). Que se repitan en la columna no viola la 1FN.

La única excepción: Solo se crearía una tabla Colours separada si el sistema necesitara llevar un inventario de códigos de pintura exactos de la fábrica (ej: ID 1 = Rosso Corsa - Hex #CC0000). En otras palabras el color se podría separar en una tabla "Colours" solo si el sistema necesita gestionar un catálogo oficial de colores (ej: "Rojo Ferrari", "Azul Última"). En este caso, como solo se necesita saber si el carro es "Rojo" o "Gris" (texto simple), se queda directamente en la tabla Cars.

La lógica del "Mundo Real" aplicada a la base de datos

Imaginando que trabajamos para una agencia de autos o un registro vehicular.

- *El Modelo (Models): Es el plano de la fábrica. Ford diseñó el "Mustang". El concepto "Mustang" no tiene un color único de fábrica (se vende en rojo, negro, azul) ni se fabricó en un solo año (se hace desde 1964 hasta el 2026).*
- *El Carro Físico (Cars): Es el auto que tiene un motor, llantas y una placa o VIN. Ese carro específico sí nació en un año exacto (ej. 2024) y se pintó de un color exacto (ej. Rojo).*

Si pusieramos el año y el color en la tabla Models, estaríamos diciendo que todos los "Mustang" del mundo son rojos y del año 2024. Y esto simplemente no es así!

Tabla MAKE

Por que "MAKE (BRAND)", tienen su propia tabla cada uno?

- Porque Make debe ser un atributo de la tabla Models (o Models debe tener una FK hacia Make). Un modelo pertenece a una marca, no el carro físico.. Ejemplo: una marca de carros por lo general patenta y fabrica sus automoviles con un nombre especifico de modelos, ejemplo, un "Accord" siempre va a pertenecer a la marca Honda, y un carro "Mustang" siempre va a corresponder a Ford.

Tabla MODELS

Porque "MODELS" tiene su propia tabla (Tabla Maestra de Identidades)?

- Porque esta funciona como tabla maestra de Entidades. El modelo define el concepto del auto diseñado por la fábrica (ej. Accord, Mustang). Aquí insertamos el nombre del modelo y el MAKE ID (FK), ya que el modelo está amarrado a una marca, pero el año y el color se quedan en la tabla Cars porque varían con cada carro físico

Tabla OWNERS

Por que "OWNERS" tiene su propia tabla?

- Porque el nombre y numero de telefono de cada cliente funciona por aparte de la informacion identitaria del carro.
- Porque en el supuesto de que una persona posea varios autos, evitamos la repeticion de su nombre en varias celdas, evitando redundancia por dependencia funcional [2FN], ya que los datos del dueño dependen de su propio ID de dueño y no del carro físico. Separar al dueño es aplicar la 2FN/3FN correctamente para que sus datos dependan solo de su PK (Owner_ID).

Tabla INSURANCE

Por que "INSURANCE" tiene una tabla por aparte?

- Porque una misma compania puede llegar a ofrecer el mismo "Insurance Policy" (Fire & Theft, Full Cover, Collision, Basic Legal)
- Porque diversos clientes pueden obtener el mismo "Insurance Policy" bajo la misma "Insurance Company".
- Porque un mismo cliente con multiples carros puede adquirir el mismo o diversos insurance policies.

Tabla CAR REGISTRATIONS

Por que un tabla para "CAR REGISTRATIONS" (Tabla Puente/Cruz)?

- Porque aqui cruzamos el "Car ID"; el cual ya incluye el model ID, con el Owner ID, creando la relacion necesaria para asociar los datos de los duenos con sus correspondientes carros, ademas de que cruzamos tambien la informacion del Insurance.